University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Robot Operating System

Srikanth Nanda

12. Dezember 2025

1 Robot Operating System (ROS)

2 Quellen

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Robot Operating System (ROS)

Overview I

Robot Operating System(ROS) is an open-source, meta-operating system for your robot. It provides the services you would expect from an operating system, including hardware abstraction, low-level device control, implementation of commonly-used functionality, message-passing between processes, and package management. It also provides tools and libraries for obtaining, building, writing, and running code across multiple computers. Quigley u. a., 2009

ROS Runtime I

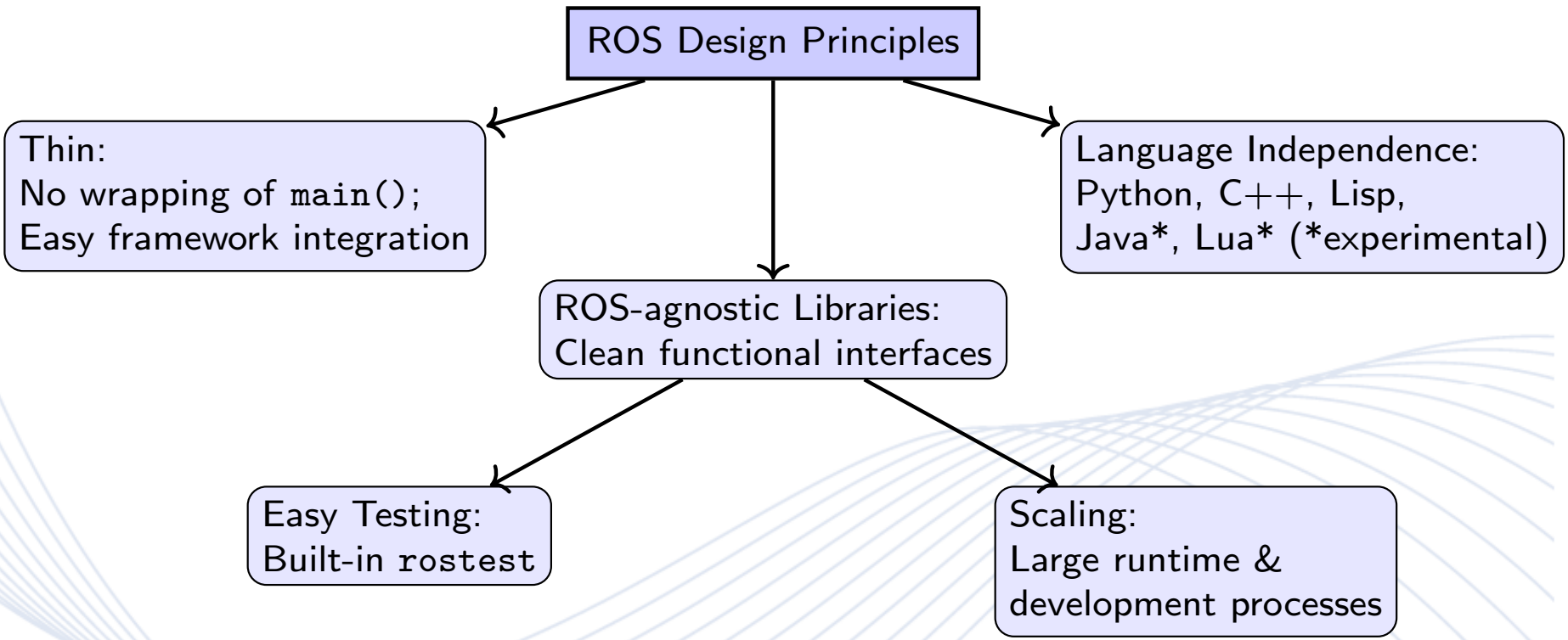
- The ROS runtime "graph" is a peer-to-peer network of processes (potentially distributed across machines) that are loosely coupled using the ROS communication infrastructure.
- ROS implements several different styles of communication, including synchronous RPC-style communication over services, asynchronous streaming of data over topics, and storage of data on a Parameter Server. These are explained in greater detail in our Conceptual Overview. Macenski u. a., 2022
- ROS is not a realtime framework, though it is possible to integrate ROS with realtime code. Quigley u. a., 2009

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Why ROS? I

- ROS is a distributed framework of processes (aka Nodes) that enables executables to be individually designed and loosely coupled at runtime. These processes can be grouped into Packages and Stacks, which can be easily shared and distributed.
- ROS also supports a federated system of code Repositories that enable collaboration to be distributed as well. Macenski u. a., 2022

ROS Design Principles



Operating System that ROS2 supports I

Software for ROS is tested on Ubuntu and Mac OS X systems, though the ROS community has been contributing support for Fedora, Gentoo, Arch Linux and other Linux platforms.Macenski u. a., 2022

ROS 2 Navigation Stack I

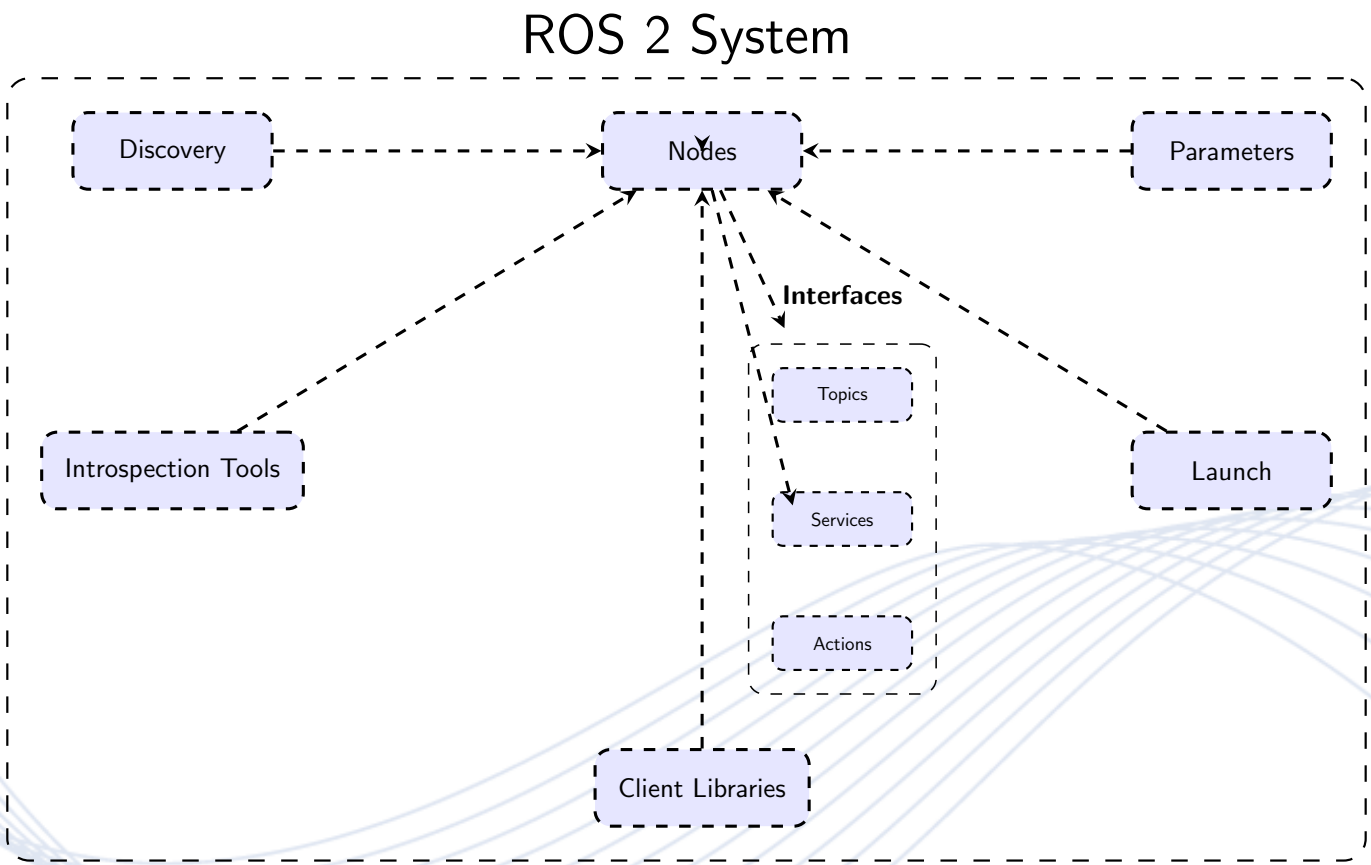
ROS 2 is a middleware based on a strongly-typed, anonymous publish and subscribe mechanism that allows for message passing between different processes.
At the core of the ROS 2 system is the ROS graph. The ROS graph refers to the network of nodes in a ROS system and the connections between them by which they communicateMacenski u. a., 2020.

Basic of ROS2 I

These are the concepts that will help you get started understanding the basics of ROS 2.

- Nodes
- Discovery
- Interfaces
- Topics
- Services
- Actions
- Parameters
- Introspection with command line tools
- Launch
- Client libraries

ROS 2 Concepts Overview



How ROS2 Communicates? I

- Nodes
  The fundamental computing units in ROS 2.
  Each node performs a specific task (e.g., sensing, planning, control).
- Discovery
  ROS 2 automatically discovers other nodes on the network.
  Allows nodes to find communication partners without manual configuration.
- Interfaces
  How nodes communicate:
  1. Topics – For continuous, asynchronous data streams (publish/subscribe).
  2. Services – For request–response interactions.
  3. Actions – For long-running tasks with feedback and the ability to cancel.

How ROS2 Communicates? II

- Parameters
  Configuration values set on nodes at runtime.
  Allow modifying node behavior without changing code.
- Client Libraries
  APIs used to write ROS 2 nodes.
  Examples: rclcpp (C++), rclpy (Python).
- Launch
  Used to start multiple nodes together with predefined configuration, parameters, and namespaces.
- Introspection Tools
  Command-line tools (e.g., ros2 topic list, ros2 node info) used to inspect what is running.
  Help debug and understand system behavior.

Nodes I

- A node is an entity that participates in the ROS 2 graph and uses a client library to interact with other nodes. Communication may occur between nodes running in the same process, in separate processes, or even on different machines.
- Nodes generally represent the fundamental units of computation in a ROS system, and each one is ideally responsible for a single, well-defined task.
- Nodes may publish data on named topics for other nodes to receive, or subscribe to topics to obtain data. They may also function as service clients, requesting computations from other nodes, or as service servers, offering capabilities to the rest of the system.

Messages I

Messages are a way for a ROS 2 node to send data on the network to other ROS nodes, with no response expected. Messages are described and defined in .msg files in the msg/ directory of a ROS package. .msg files are composed of two parts: fields and constants. Macenski u. a., 2023
Each field consists of a type and a name, separated by a space, i.e:

- `fieldtype1 fieldname1`
- `fieldtype2 fieldname2`
- `fieldtype3 fieldname3`
- `int32 myint`
- `string mystring`

Field types can be:

1. a built-in-type
2. names of Message descriptions defined on their own, such as `geometry_msgs/PoseStamped`

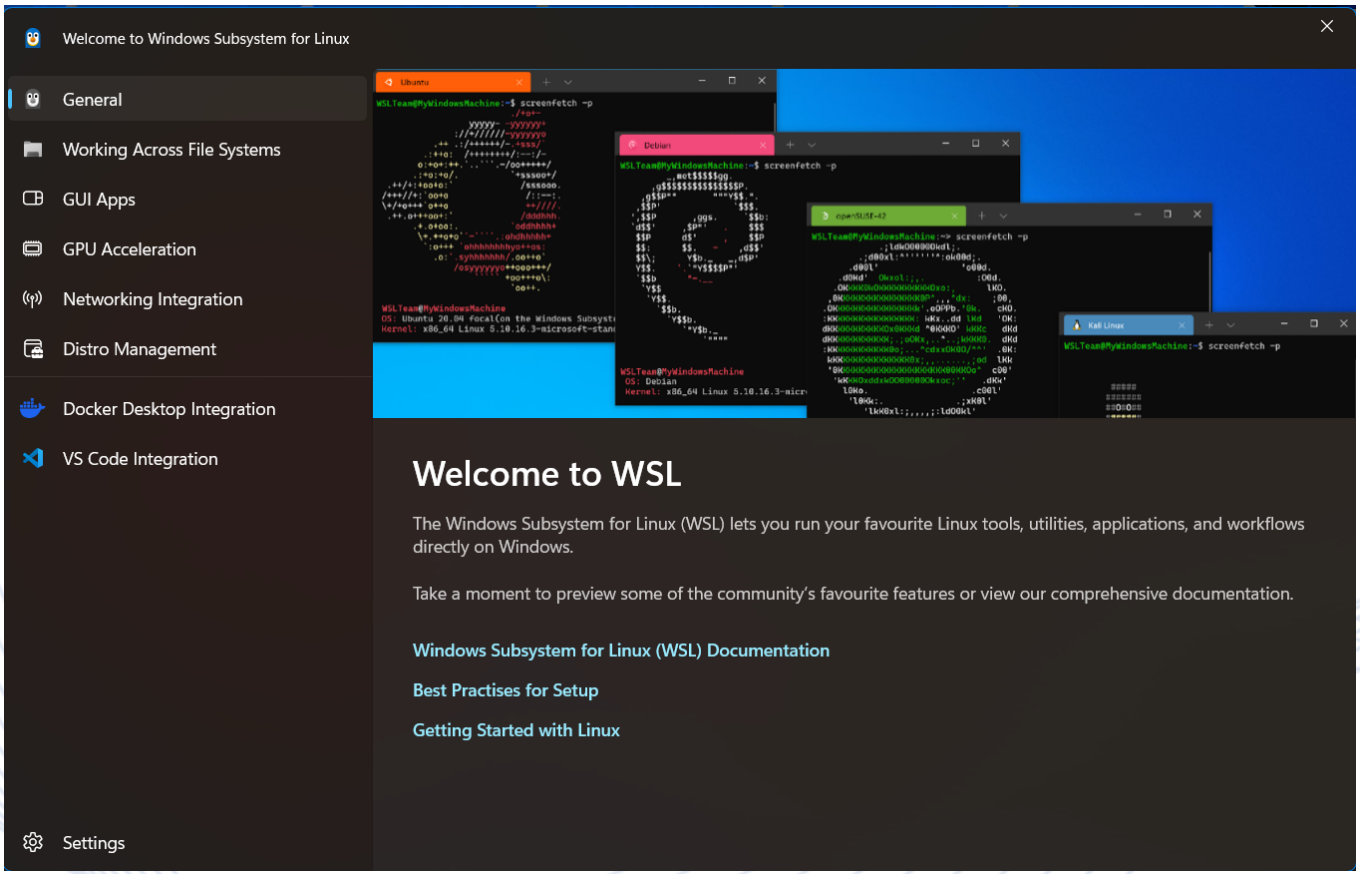
ROS 2 AMCL I

amcl is a probabilistic localization system for a robot moving in 2D. It implements the adaptive (or KLD-sampling) Monte Carlo localization approach (as described by Dieter Fox), which uses a particle filter to track the pose of a robot against a known map.Macenski u. a., 2022 This node is derived, with thanks, from Andrew Howard's excellent 'amcl' Player driver.

Robot Localization I

[robot_localization] robot_localization is a package of nonlinear state estimation nodes. The package was developed by Charles River Analytics, Inc.
Fuse the odometry X and Y velocities only, not the poses. Turn off differential. Tune covariances to be proportational to your confidence in that measurement. For VO, decrease your confidence in the XY pose significantly. Macenski u. a., 2023

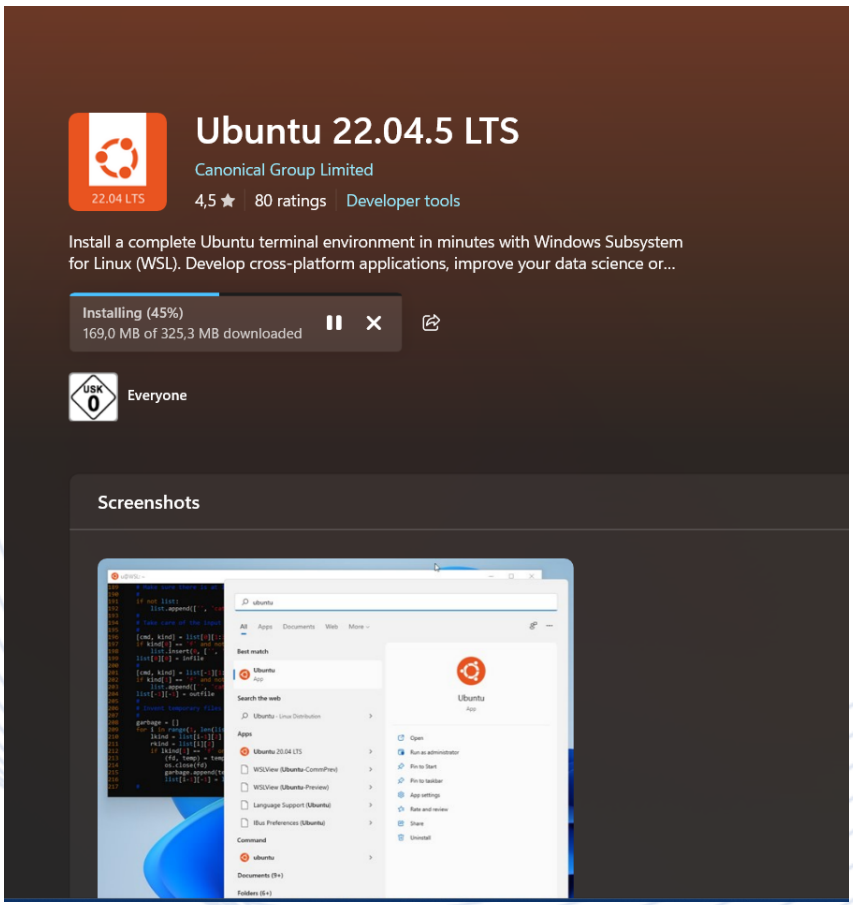
WSL welcome interface I



[1]indows Subsystem for Linux startup screen presenting available features and integration capabilities.

[1]W

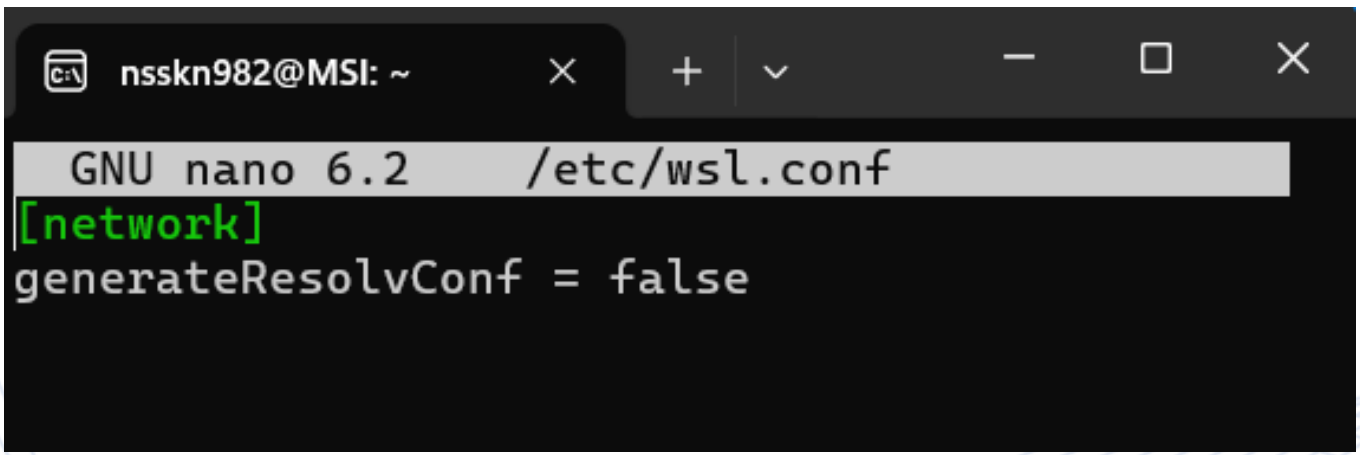
Ubuntu installation from Microsoft Store I



[2]

[2] Installing Ubuntu 22.04 LTS from the Microsoft Store for use with WSL.

WSL DNS configuration file I



[3]

[3] Editing `/etc/wsl.conf` to disable automatic resolv.conf generation for improved DNS stability.

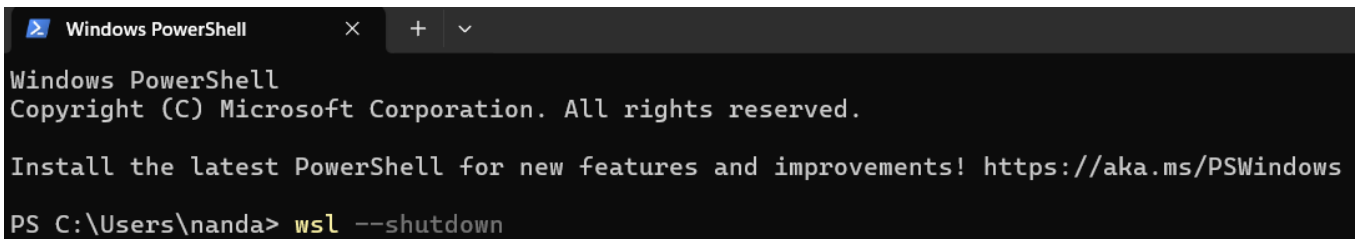
Editing resolv.conf and wsl.conf I

```
nsskn982@MSI:~$ sudo nano /etc/resolv.conf
nsskn982@MSI:~$ sudo nano /etc/wsl.conf
```

[4]

[4] Commands used to manually adjust WSL resolver settings and networking configuration.

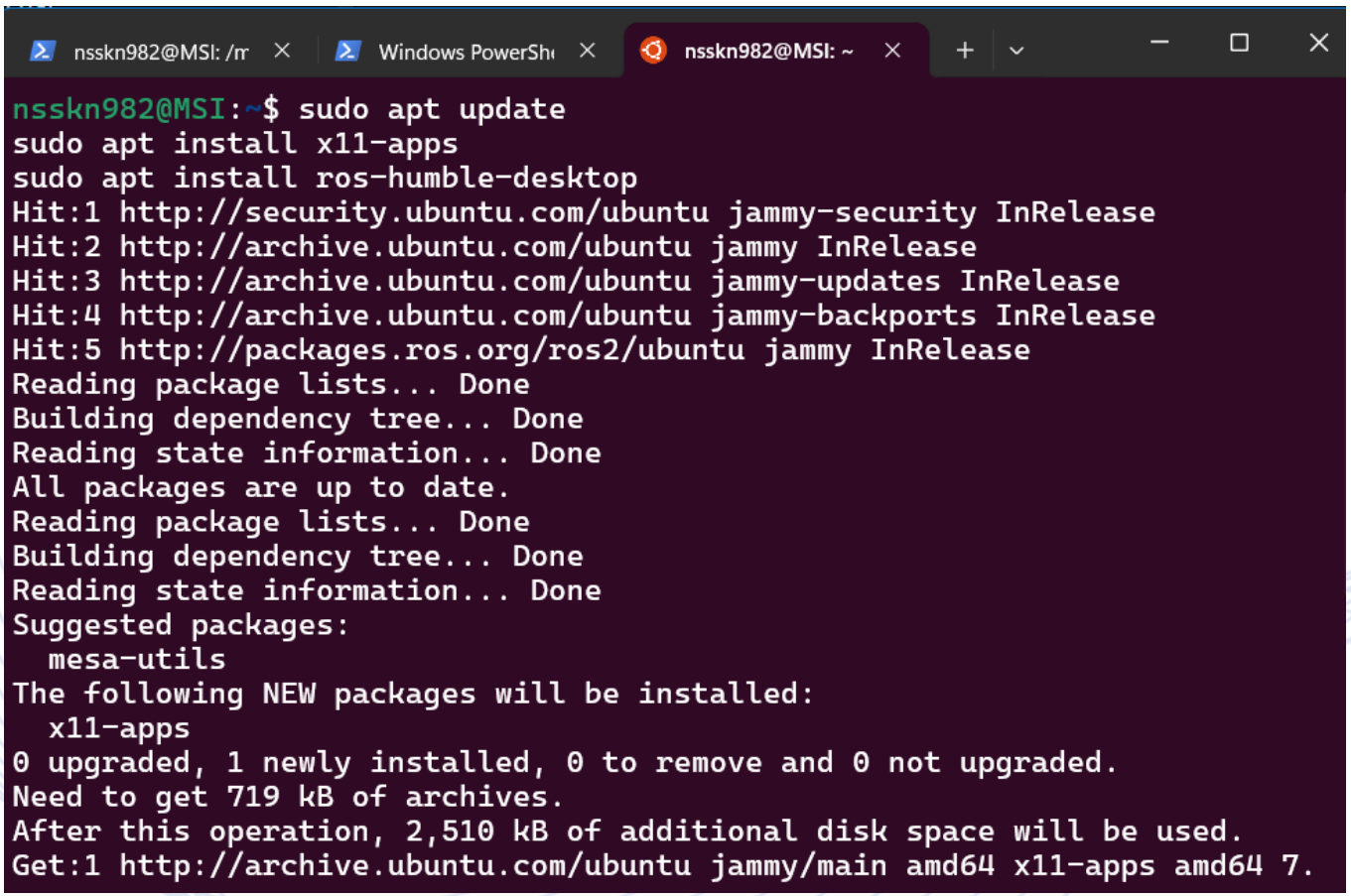
Shutting down WSL environment I



5

[5] Executing `wsl --shutdown` in PowerShell to restart the subsystem after configuration changes.

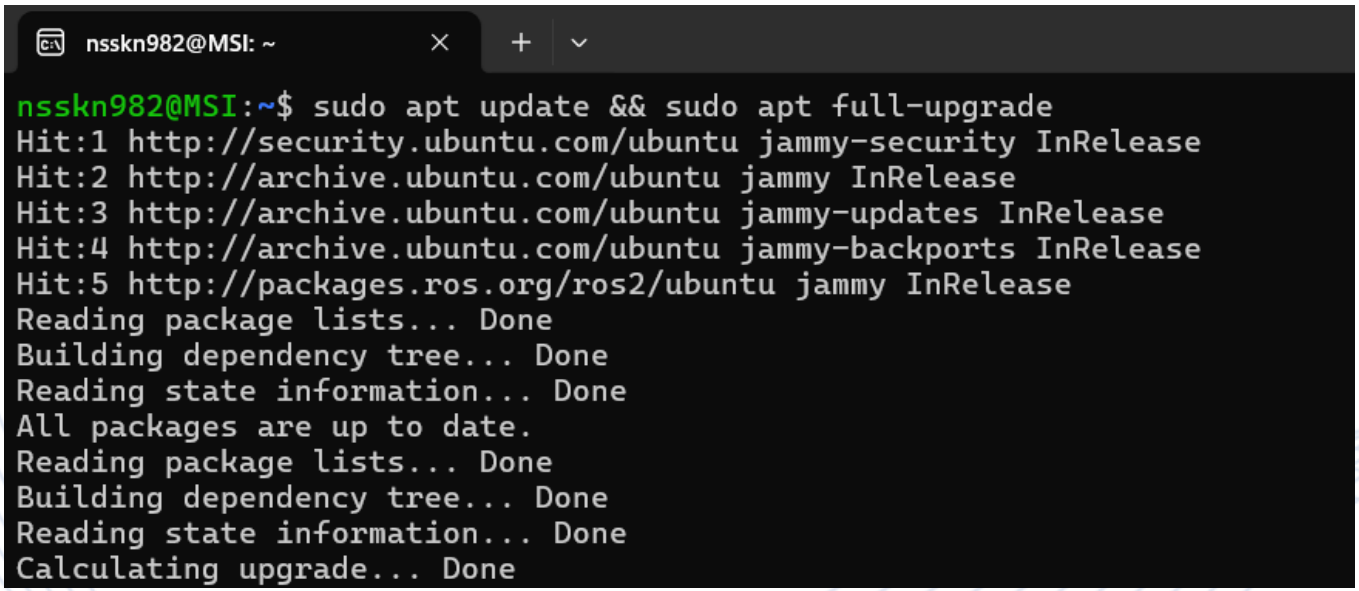
Updating APT package lists I



[6]

[6] Refreshing system package indexes before installing ROS 2 and related dependencies.

Full system upgrade I

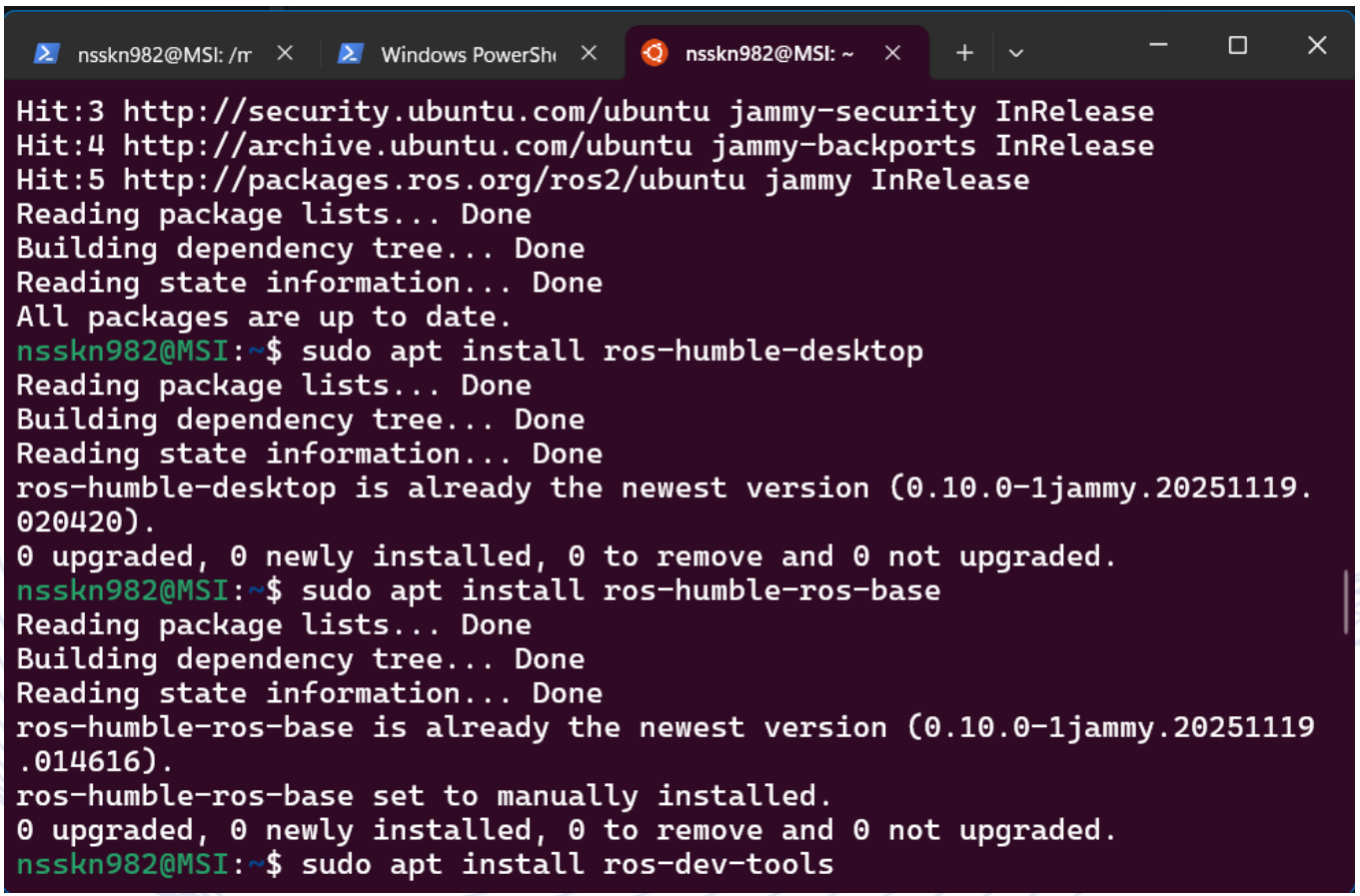


[7]

[7] Performing a complete system upgrade to ensure all libraries are current.

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Installing ROS 2 desktop and base packages I



[8]

[8] Output confirming installation of ROS 2 Humble desktop, core, and supporting development tools.

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

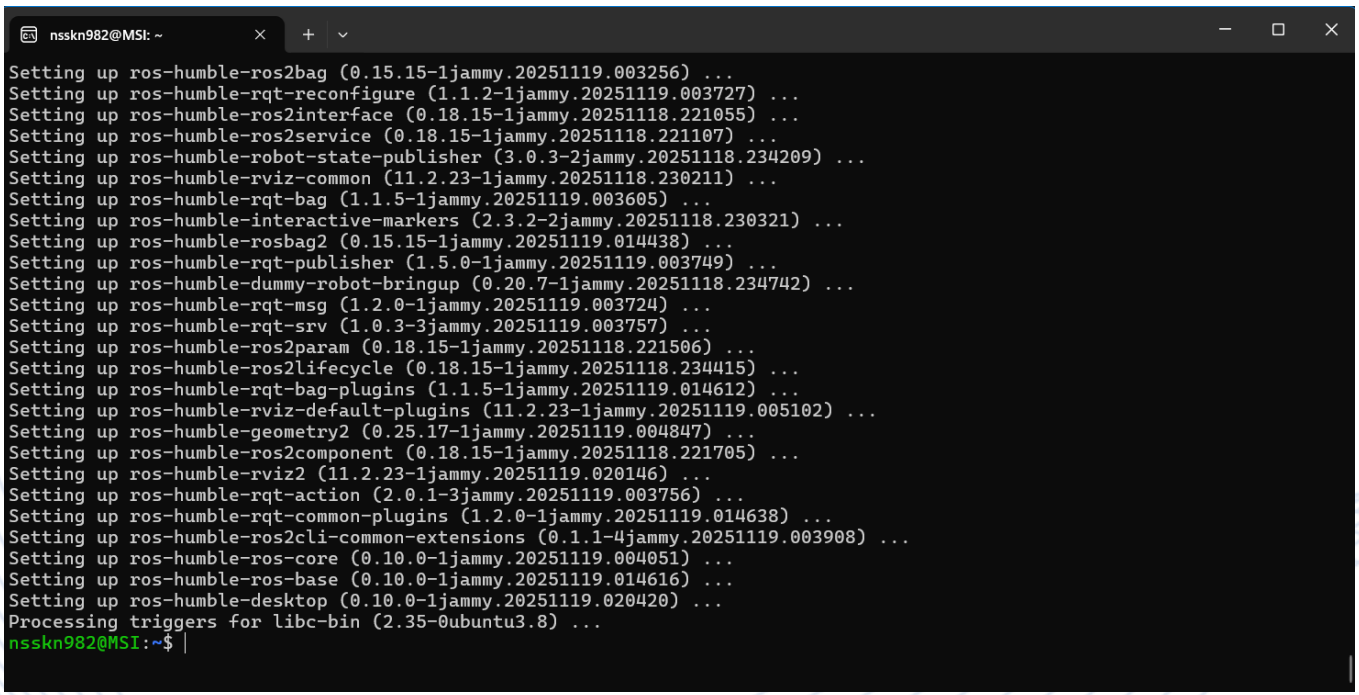
Installing rqt and GUI plugins I

```
nsskn982@MSI:~$ sudo apt update
sudo apt install '~nros-humble-rqt*'
[sudo] password for nsskn982:
Hit:1 https://brave-browser-apt-release.s3.brave.com stable InRelease
Hit:2 http://archive.ubuntu.com/ubuntu jammy InRelease
Hit:3 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:4 http://archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:5 http://packages.ros.org/ros2/ubuntu jammy InRelease
Hit:6 http://archive.ubuntu.com/ubuntu jammy-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
5 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ros-humble-rqt-action is already the newest version (2.0.1-3jammy.20251119.003756).
ros-humble-rqt-action set to manually installed.
ros-humble-rqt-bag is already the newest version (1.1.5-1jammy.20251119.003605).
ros-humble-rqt-bag set to manually installed.
ros-humble-rqt-bag-plugins is already the newest version (1.1.5-1jammy.20251119.014612).
ros-humble-rqt-bag-plugins set to manually installed.
ros-humble-rqt-common-plugins is already the newest version (1.2.0-1jammy.20251119.014638).
ros-humble-rqt-common-plugins set to manually installed.
ros-humble-rqt-console is already the newest version (2.0.3-1jammy.20251119.003629).
ros-humble-rqt-console set to manually installed.
ros-humble-rqt-graph is already the newest version (1.3.1-1jammy.20251119.003747).
ros-humble-rqt-graph set to manually installed.
ros-humble-rqt-gui is already the newest version (1.1.9-1jammy.20251118.220921).
ros-humble-rqt-gui set to manually installed.
ros-humble-rqt-gui-cpp is already the newest version (1.1.9-1jammy.20251118.234640).
ros-humble-rqt-gui-cpp set to manually installed.
ros-humble-rqt-gui-py is already the newest version (1.1.9-1jammy.20251119.003313).
ros-humble-rqt-gui-py set to manually installed.
ros-humble-rqt-image-view is already the newest version (1.2.0-2jammy.20251118.235106).
ros-humble-rqt-image-view set to manually installed.
ros-humble-rqt-msg is already the newest version (1.2.0-1jammy.20251119.003724).
ros-humble-rqt-msg set to manually installed.
```

[9]

[9] Installation of rqt and visualization plugins used for debugging and introspection.

Installing ROS dependency tools I



[10]

[10] Additional ROS 2 Humble tools and libraries configured inside the WSL environment.

Fetching updated ROS 2 Humble repository file I

```
nsskn982@MSI:~$ cd ~/ros2_humble
mv -i ros2.repos ros2.repos.old
wget https://raw.githubusercontent.com/ros2/ros2/humble/ros2.repos
-bash: cd: /home/nsskn982/ros2_humble: No such file or directory
mv: cannot stat 'ros2.repos': No such file or directory
--2025-12-09 12:13:48--  https://raw.githubusercontent.com/ros2/ros2/humble/ros2
.repos
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.109.1
33, 185.199.111.133, 185.199.108.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.109.
133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 11667 (11K) [text/plain]
Saving to: ‘ros2.repos’

ros2.repos           100%[=====================>]  11.39K  --.-KB/s    in 0s

2025-12-09 12:13:48 (85.7 MB/s) - ‘ros2.repos’ saved [11667/11667]
```

[11]

[11] Downloading the latest `ros2.repos` file for workspace synchronization.

ROS environment variables after setup I

```
nsskn982@MSI:~$ echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
nsskn982@MSI:~$ printenv | grep -i ROS
ROS_VERSION=2
ROS_PYTHON_VERSION=3
AMENT_PREFIX_PATH=/opt/ros/humble
PYTHONPATH=/opt/ros/humble/lib/python3.10/site-packages:/opt/ros/humble/local/lib/python3.10/dist-packages
LD_LIBRARY_PATH=/usr/lib/x86_64-linux-gnu/gazebo-11/plugins:/opt/ros/humble/opt/rviz_ogre_vendor/lib:/opt/ros/
humble/lib/x86_64-linux-gnu:/opt/ros/humble/lib
ROS_LOCALHOST_ONLY=0
PATH=/home/nsskn982/.local/bin:/opt/ros/humble/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bi
n:/usr/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/WINDOWS/system32:/mnt/c/WINDOWS:/mnt/c/WINDOWS/System32/
Wbem:/mnt/c/WINDOWS/System32/WindowsPowerShell/v1.0/:/mnt/c/WINDOWS/System32/OpenSSH/:/mnt/c/Program Files (x8
6)/NVIDIA Corporation/PhysX/Common:/mnt/c/Program Files/doxygen/bin:/mnt/c/Program Files/Graphviz/bin:/mnt/c/P
rogram Files/NVIDIA Corporation/NVIDIA app/NvDLISR:/mnt/c/Program Files/Arduino CLI/:/mnt/c/Program Files/PuTT
Y/:/mnt/c/Program Files/nodejs/:/mnt/c/Program Files/NVIDIA Corporation/SDK Manager:/mnt/c/Program Files/Docke
r/Docker/resources/bin:/mnt/c/Users/nanda/AppData/Local/Programs/Python/Python311/Scripts/:/mnt/c/Users/nanda/
AppData/Local/Programs/Python/Python311/:/mnt/c/Program Files/MySQL/MySQL Shell 8.0/bin/:/mnt/c/Users/nanda/Ap
pData/Local/Programs/Python/Launcher/:/mnt/c/Users/nanda/AppData/Local/Microsoft/WindowsApps:/mnt/c/Users/nand
a/AppData/Local/Programs/Microsoft VS Code/bin:/mnt/c/Program Files/JetBrains/PyCharm 2024.2.4/bin:/mnt/c/User
s/nanda/AppData/Local/GitHubDesktop/bin:/mnt/c/Users/nanda/AppData/Local/Programs/Git/cmd:/mnt/c/Users/nanda/A
ppData/Local/Pandoc/:/mnt/c/texlive/2024/bin/windows:/mnt/c/Program Files/JetBrains/PyCharm Community Edition
2024.3.1/bin:/mnt/c/Users/nanda/AppData/Roaming/npm:/snap/bin
ROS_DISTRO=humble
```

[12]

[12] Listing ROS-specific environment variables such as paths, Python version, and distro identifier.

Checking ROS distribution I

```
nsskn982@MSI:~$ echo $ROS_DISTRO
humble
```

[13]

[13] Verifying that the distribution variable `ROS_DISTRO` is correctly set to Humble.

Checking active RMW implementation I

```
nsskn982@MSI:~$ ros2 doctor --report | grep RMW
    RMW MIDDLEWARE
```

[14]

[14] Querying the ROS 2 middleware layer using `ros2doctor`.

Setting a valid ROS_DOMAIN_ID I

```
nsskn982@MSI:~$ ros2 pkg executables turtlesim
turtlesim draw_square
turtlesim mimic
turtlesim turtle_teleop_key
turtlesim turtlesim_node
nsskn982@MSI:~$ ros2 run turtlesim turtlesim_node
Calculated port number is too high. Probably the domainId is over
232 or portBase is too high.
2025-12-09 10:39:00.887 [RTPS Error] Calculated port number is too
 high. Probably the domainId is over 232 or portBase is too high.
-> Function getMulticastPort
[ros2run]: Process exited with failure 1
nsskn982@MSI:~$ export ROS_DOMAIN_ID=111
nsskn982@MSI:~$ echo "export ROS_DOMAIN_ID=111" >> ~/.bashrc
```

15

[15] Applying and persisting a corrected ROS_DOMAIN_ID to avoid port conflicts.

C++ talker node executing I

```
nsskn982@MSI:~$ echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
source ~/.bashrc
nsskn982@MSI:~$ source /opt/ros/humble/setup.bash
ros2 run demo_nodes_cpp talker
[INFO] [1765027269.491294102] [talker]: Publishing: 'Hello World: 1'
[INFO] [1765027270.517689023] [talker]: Publishing: 'Hello World: 2'
[INFO] [1765027271.544217648] [talker]: Publishing: 'Hello World: 3'
[INFO] [1765027272.570615806] [talker]: Publishing: 'Hello World: 4'
[INFO] [1765027273.595268583] [talker]: Publishing: 'Hello World: 5'
[INFO] [1765027274.620452205] [talker]: Publishing: 'Hello World: 6'
[INFO] [1765027275.645520839] [talker]: Publishing: 'Hello World: 7'
[INFO] [1765027276.670595235] [talker]: Publishing: 'Hello World: 8'
[INFO] [1765027276.948419743] [talker]: Publishing: 'Hello World: 9'
[INFO] [1765027277.973415172] [talker]: Publishing: 'Hello World: 10'
[INFO] [1765027278.998488542] [talker]: Publishing: 'Hello World: 11'
[INFO] [1765027280.023545283] [talker]: Publishing: 'Hello World: 12'
```

16

[16] Demo publisher repeatedly broadcasting messages using the C++ ROS 2 talker example.

Python listener node output I

```
nsskn982@MSI:~$ ros2 run demo_nodes_py listener
[INFO] [1765027446.612932771] [listener]: I heard: [Hello World: 1]
[INFO] [1765027447.620995883] [listener]: I heard: [Hello World: 2]
[INFO] [1765027448.646196341] [listener]: I heard: [Hello World: 3]
[INFO] [1765027449.672057788] [listener]: I heard: [Hello World: 4]
[INFO] [1765027450.697114715] [listener]: I heard: [Hello World: 5]
[INFO] [1765027451.722276489] [listener]: I heard: [Hello World: 6]
[INFO] [1765027452.747514053] [listener]: I heard: [Hello World: 7]
[INFO] [1765027453.036518801] [listener]: I heard: [Hello World: 8]
[INFO] [1765027454.061658055] [listener]: I heard: [Hello World: 9]
[INFO] [1765027455.087092474] [listener]: I heard: [Hello World: 10]
[INFO] [1765027456.112549033] [listener]: I heard: [Hello World: 11]
[INFO] [1765027457.137425318] [listener]: I heard: [Hello World: 12]
[INFO] [1765027458.163931421] [listener]: I heard: [Hello World: 13]
[INFO] [1765027459.187974419] [listener]: I heard: [Hello World: 14]
[INFO] [1765027460.213347512] [listener]: I heard: [Hello World: 15]
[INFO] [1765027461.238848582] [listener]: I heard: [Hello World: 16]
[INFO] [1765027462.264462073] [listener]: I heard: [Hello World: 17]
[INFO] [1765027463.289616603] [listener]: I heard: [Hello World: 18]
[INFO] [1765027464.314993774] [listener]: I heard: [Hello World: 19]
```

17

[17] Listening to received messages from the talker node, confirming functional pub/sub transport.

Installing turtlesim package I

```
nsskn982@MSI:~$ sudo apt update
sudo apt install ros-humble-turtlesim
[sudo] password for nsskn982:
Hit:1 http://archive.ubuntu.com/ubuntu jammy InRelease
Get:2 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Hit:3 https://brave-browser-apt-release.s3.brave.com stable InRelease
Get:4 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Get:5 http://archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Get:6 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [3,110 kB]
Hit:7 http://packages.ros.org/ros2/ubuntu jammy InRelease
Get:8 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages [2,850 kB]
Get:9 http://archive.ubuntu.com/ubuntu jammy-updates/main Translation-en [478 kB]
Get:10 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages [1,244 kB]
Get:11 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 c-n-f Metadata [30.0 kB]
Get:12 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages [1,006 kB]
Get:13 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 c-n-f Metadata [22.3 kB]
Fetched 9,125 kB in 2s (4,682 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
5 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ros-humble-turtlesim is already the newest version (1.4.3-1jammy.20251118.232100).
```

18

[18] Installing the classic turtlesim simulator used for basic ROS messaging demonstrations.

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

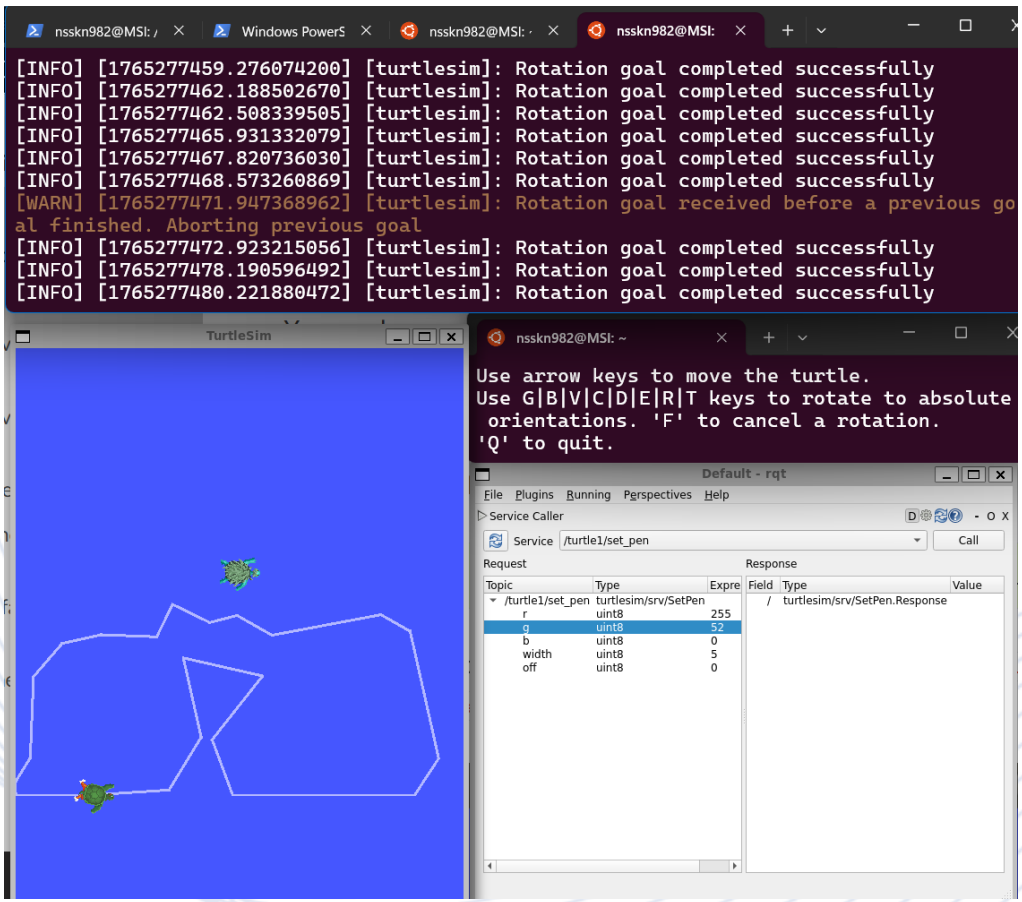
Running turtlesim with rqt integration I

```
turtlesim draw_square
turtlesim mimic
turtlesim turtle_teleop_key
turtlesim turtlesim_node
nsskn982@MSI:~$ ros2 run turtlesim turtlesim_node
Calculated port number is too high. Probably the domainId is over
232 or portBase is too high.
2025-12-09 10:39:00.887 [RTPS Error] Calculated port number is too
 high. Probably the domainId is over 232 or portBase is too high.
-> Function getMulticastPort
[ros2run]: Process exited with failure 1
nsskn982@MSI:~$ export ROS_DOMAIN_ID=111
nsskn982@MSI:~$ echo "export ROS_DOMAIN_ID=111" >> ~/.bashrc
nsskn982@MSI:~$ ros2 run turtlesim turtlesim_node
[INFO] [1765273270.618716214] [turtlesim]: Starting turtlesim with
 node name /turtlesim
[INFO] [1765273270.629514446] [turtlesim]: Spawning turtle [turtle
1] at x=[5.544445], y=[5.544445], theta=[0.000000]
^Z
[1]+  Stopped                 ros2 run turtlesim turtlesim_node
```

19

[19] Operating turtlesim alongside rqt_graph to visualize node connectivity and topic flow.

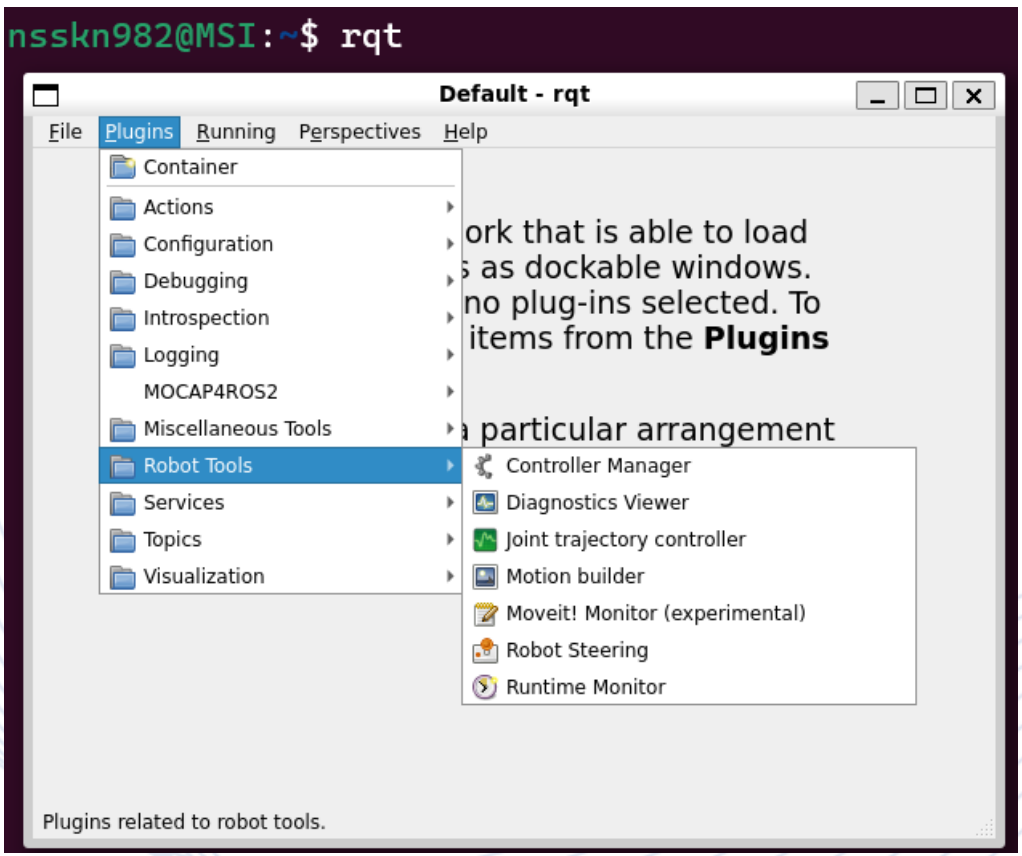
Turtle tele-operation and pen control I



20

[20] Teleoperation with keyboard input and service calls altering drawing parameters.

Available rqt plugins I



[21]

[21] Plugin browser showing rqt tools for visualization, debugging, and diagnostics.

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Topic list with message types I

```
nsskn982@MSI:~$ ros2 topic list -t
/parameter_events [rcl_interfaces/msg/ParameterEvent]
/rosout [rcl_interfaces/msg/Log]
/turtle1/cmd_vel [geometry_msgs/msg/Twist]
/turtle1/color_sensor [turtlesim/msg/Color]
/turtle1/pose [turtlesim/msg/Pose]
```

22

[22] Enumeration of active topics with corresponding message types.

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

ros2 topic command overview I

```
nsskn982@MSI:~$ ros2 topic -h
usage: ros2 topic [-h] [--include-hidden-topics] Call `ros2 topic <command> -h` for more detailed usage. ...

Various topic related sub-commands

options:
  -h, --help            show this help message and exit
  --include-hidden-topics
                        Consider hidden topics as well

Commands:
  bw     Display bandwidth used by topic
  delay  Display delay of topic from timestamp in header
  echo   Output messages from a topic
  find   Output a list of available topics of a given type
  hz     Print the average receiving rate to screen
  info   Print information about a topic
  list   Output a list of available topics
  pub    Publish a message to a topic
  type   Print a topic's type
```

23

[23] Help listing of topic management commands for introspection and publishing.

ros2 node command overview I

```
nsskn982@MSI:~$ ros2 node -h
usage: ros2 node [-h] Call `ros2 node <command> -h` for more detailed usage. ...

Various node related sub-commands

options:
  -h, --help            show this help message and exit

Commands:
  info  Output information about a node
  list  Output a list of available nodes

  Call `ros2 node <command> -h` for more detailed usage.
```

24

[24] Summary of node inspection commands used for graph-level debugging.

ros2 service command overview I

```
nsskn982@MSI:~$ ros2 service -h
usage: ros2 service [-h] [--include-hidden-services] Call `ros2 service <command> -h` for more detailed usage. ...

Various service related sub-commands

options:
  -h, --help            show this help message and exit
  --include-hidden-services
                        Consider hidden services as well

Commands:
  call  Call a service
  find  Output a list of available services of a given type
  list  Output a list of available services
  type  Output a service's type

  Call `ros2 service <command> -h` for more detailed usage.
```

[25]

[25] Services-related command suite for querying and invoking server endpoints.

ros2 action command overview I

```
nsskn982@MSI:~$ ros2 action -h
usage: ros2 action [-h] Call `ros2 action <command> -h` for more detailed usage. ...

Various action related sub-commands

options:
  -h, --help            show this help message and exit

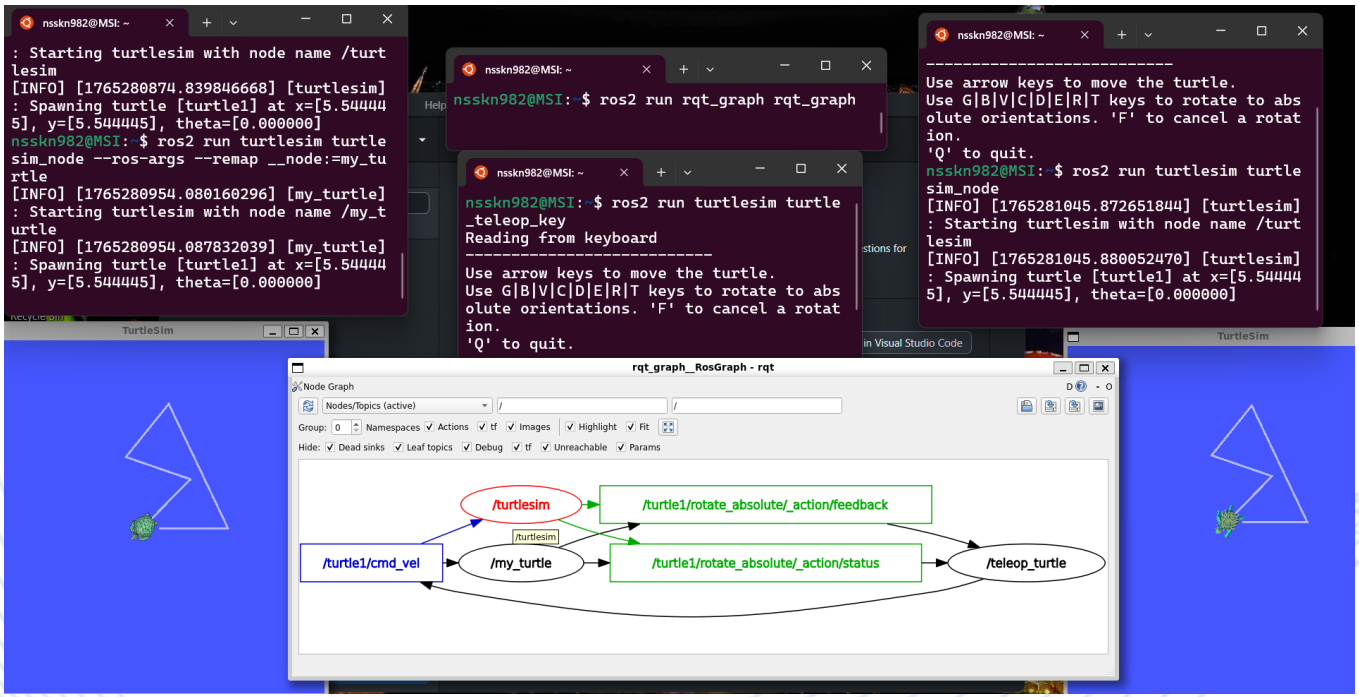
Commands:
  info       Print information about an action
  list       Output a list of action names
  send_goal  Send an action goal

  Call `ros2 action <command> -h` for more detailed usage.
```

26

[26] Commands for listing, inspecting, and executing ROS 2 action goals.

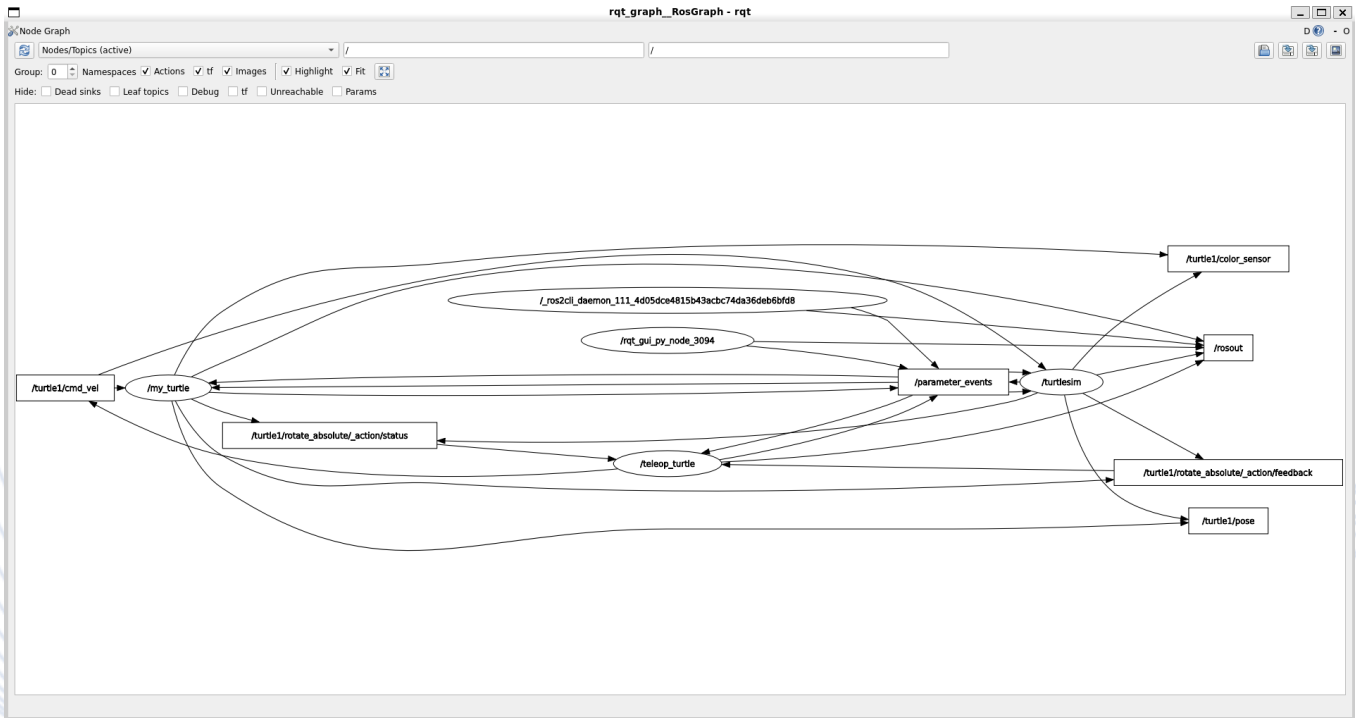
rqt graph with two nodes I



27

[27] Graph representation of a remapped turtlesim node and associated communication channels.

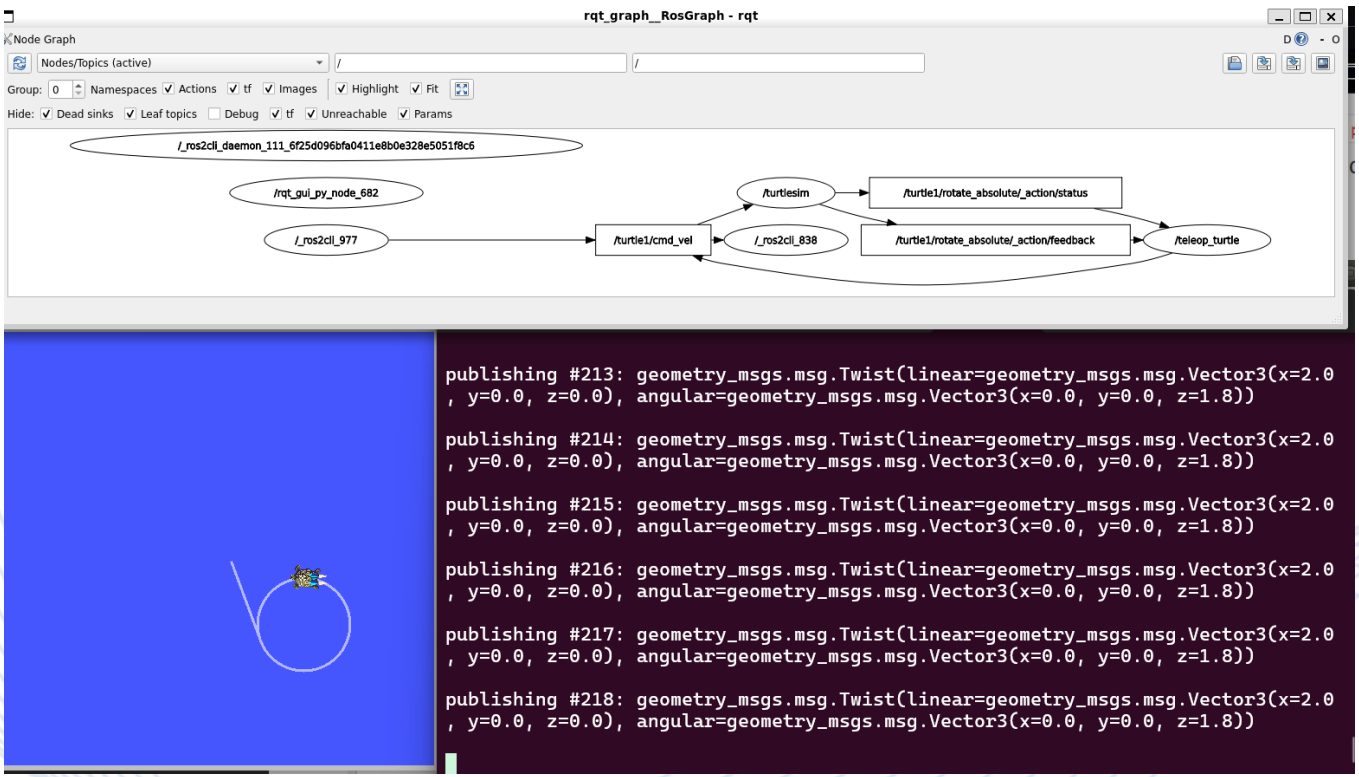
Expanded rqt graph full mapping I



28

[28] Complete topic-node map showing all active communication pathways.

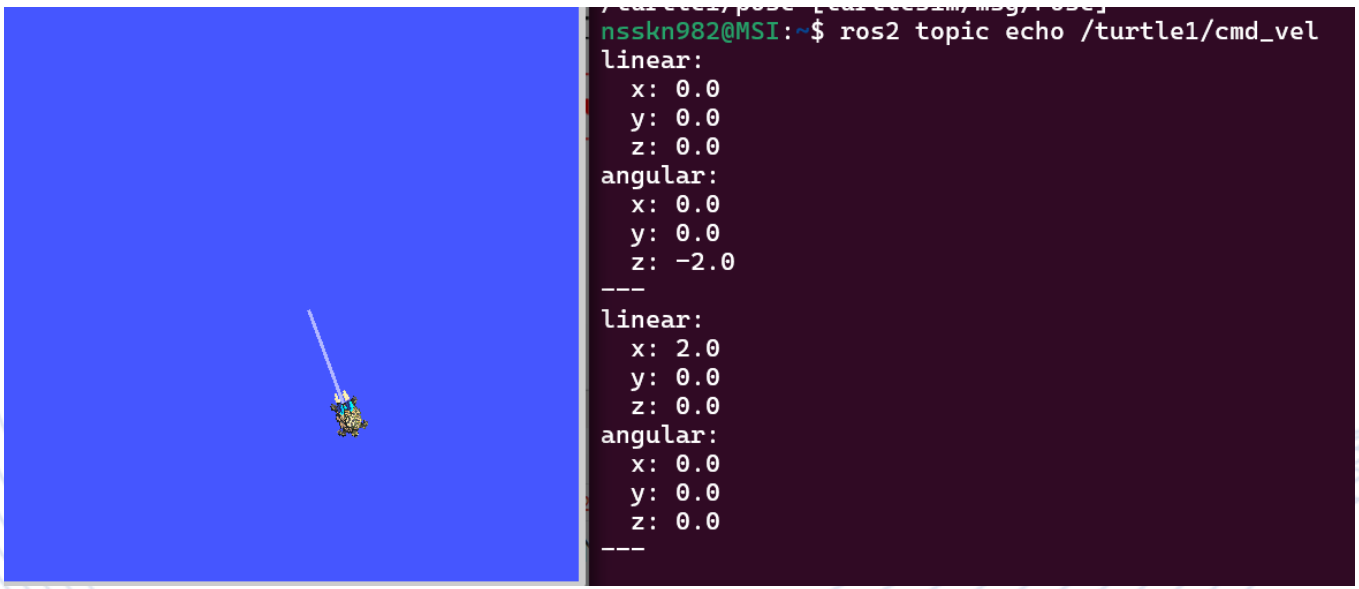
Circular motion rqt graph I



29

[29] Visualization of node interactions during scripted turtle movement in a closed curve.

Velocity command topic output I



[30]

[30] Live data from the `/turtle1/cmd_vel` topic showing commanded velocities.

Installing rmw_zenoh backend I

```
nsskn982@MSI:/home$ sudo apt install ros-humble-rmw-zenoh-cpp
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  ros-humble-zenoh-cpp-vendor
The following NEW packages will be installed:
  ros-humble-rmw-zenoh-cpp ros-humble-zenoh-cpp-vendor
0 upgraded, 2 newly installed, 0 to remove and 7 not upgraded.
Need to get 9,829 kB of archives.
After this operation, 48.5 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://packages.ros.org/ros2/ubuntu jammy/main amd64 ros-humble-zenoh-cpp-
vendor amd64 0.1.8-1jammy.20251114.103733 [9,641 kB]
Get:2 http://packages.ros.org/ros2/ubuntu jammy/main amd64 ros-humble-rmw-zenoh-
cpp amd64 0.1.8-1jammy.20251114.110222 [189 kB]
Fetched 9,829 kB in 6s (1,513 kB/s)
```

31

[31] Adding the Zenoh-based RMW implementation for alternative DDS-free communication.

Installing TurtleBot3 packages I

```
nsskn982@MSI: $ sudo apt install ros-humble-turtlebot3*
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Note, selecting 'ros-humble-turtlebot3-manipulation-teleop' for glob 'ros-humble-turtlebot3*'
Note, selecting 'ros-humble-turtlebot3-autorace-camera' for glob 'ros-humble-turtlebot3*'
Note, selecting 'ros-humble-turtlebot3-home-service-challenge-aruco' for glob 'ros-humble-turtlebot3*'
Note, selecting 'ros-humble-turtlebot3-follower' for glob 'ros-humble-turtlebot3*'
Note, selecting 'ros-humble-turtlebot3-home-service-challenge-tools' for glob 'ros-humble-turtlebot3*'
Note, selecting 'ros-humble-turtlebot3-fake-node' for glob 'ros-humble-turtlebot3*'
Note, selecting 'ros-humble-turtlebot3-manipulation-moveit-config' for glob 'ros-humble-turtlebot3*'
Note, selecting 'ros-humble-turtlebot3-example' for glob 'ros-humble-turtlebot3*'
Note, selecting 'ros-humble-turtlebot3-home-service-challenge' for glob 'ros-humble-turtlebot3*'
Note, selecting 'ros-humble-turtlebot3-manipulation-hardware' for glob 'ros-humble-turtlebot3*'
Note, selecting 'ros-humble-turtlebot3-manipulation-navigation2' for glob 'ros-humble-turtlebot3*'
Note, selecting 'ros-humble-turtlebot3-autorace' for glob 'ros-humble-turtlebot3*'
Note, selecting 'ros-humble-turtlebot3-fake-node-dbgsym' for glob 'ros-humble-turtlebot3*'
Note, selecting 'ros-humble-turtlebot3-cartographer' for glob 'ros-humble-turtlebot3*'
Note, selecting 'ros-humble-turtlebot3-home-service-challenge-manipulator' for glob 'ros-humble-turtlebot3*'
Note, selecting 'ros-humble-turtlebot3-bringup' for glob 'ros-humble-turtlebot3*'
Note, selecting 'ros-humble-turtlebot3-node-dbgsym' for glob 'ros-humble-turtlebot3*'
Note, selecting 'ros-humble-turtlebot3-manipulation' for glob 'ros-humble-turtlebot3*'
Note, selecting 'ros-humble-turtlebot3-applications-msgs-dbgsym' for glob 'ros-humble-turtlebot3*'
Note, selecting 'ros-humble-turtlebot3-panorama-dbgsym' for glob 'ros-humble-turtlebot3*'
Note, selecting 'ros-humble-turtlebot3-home-service-challenge-manipulator-dbgsym' for glob 'ros-humble-turtlebot3*'
Note, selecting 'ros-humble-turtlebot3-msgs' for glob 'ros-humble-turtlebot3*'
Note, selecting 'ros-humble-turtlebot3-manipulation-teleop-dbgsym' for glob 'ros-humble-turtlebot3*'
Note, selecting 'ros-humble-turtlebot3-automatic-parking' for glob 'ros-humble-turtlebot3*'
Note, selecting 'ros-humble-turtlebot3-autorace-detect' for glob 'ros-humble-turtlebot3*'
Note, selecting 'ros-humble-turtlebot3-yolo-object-detection' for glob 'ros-humble-turtlebot3*'
Note, selecting 'ros-humble-turtlebot3-manipulation-cartographer' for glob 'ros-humble-turtlebot3*'
```

[32]

[32] Automatic selection and installation of TurtleBot3 modules for ROS 2 Humble.

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

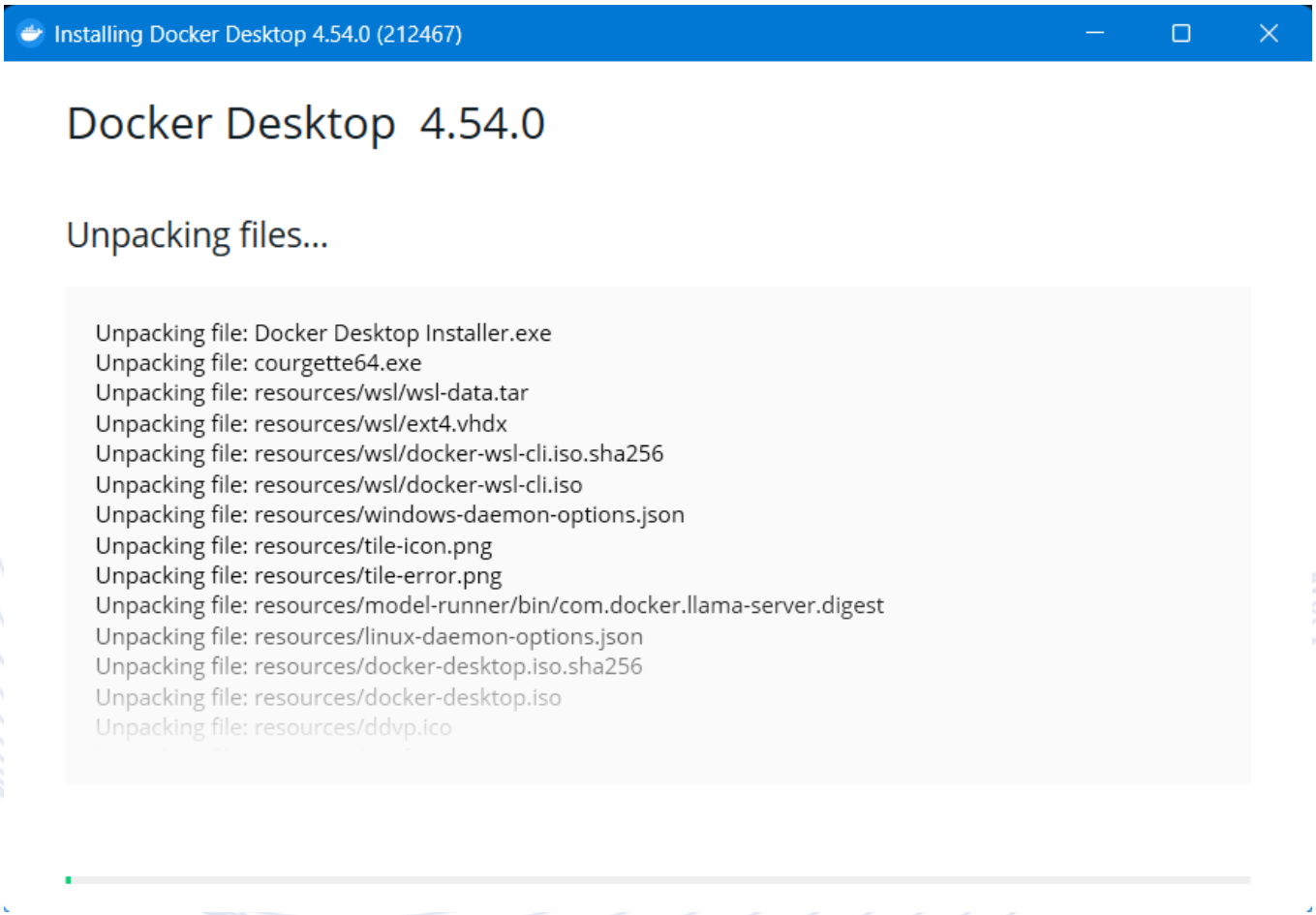
Installing Navigation 2 (Nav2) stack I

```
nsskn982@MSI:~$ sudo apt install ros-humble-navigation2 ros-humble-nav2-bringup
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following package was automatically installed and is no longer required:
  python3-rospkg
Use 'sudo apt autoremove' to remove it.
The following additional packages will be installed:
  bzip2-doc comerr-dev graphicsmagick-libmagick-dev-compat krb5-multidev libbenchmark-dev libbenchmark1
  libbsd-dev libbz2-dev libceres-dev libceres2 libgflags-dev libgflags2.2 libgoogle-glog-dev
  libgoogle-glog0v5 libgraphblas-dev libgraphblas6 libgraphics-magick-perl libgraphicsmagick++1-dev
  libgraphicsmagick1-dev libgssrpc4 libkadm5clnt-mit12 libkadm5srv-mit12 libkdb5-10 libkrb5-dev liblcms2-dev
  libldl2 libmd-dev libmongoose2 libnanoflann-dev libncurses-dev libnorm-dev libomp-14-dev libomp-dev
  libomp5-14 libpgm-dev librbio2 libsliplu1 libsodium-dev libspqr2 libsuitesparse-dev libunwind-dev
  libwmf-0.2-7 libwmf-dev libxtensor-dev libzmq3-dev nlohmann-json3-dev ros-humble-behaviortree-cpp-v3
  ros-humble-bond ros-humble-bondcpp ros-humble-costmap-queue ros-humble-diagnostic-updater
  ros-humble-dwb-core ros-humble-dwb-critics ros-humble-dwb-msgs ros-humble-dwb-plugins
  ros-humble-nav-2d-msgs ros-humble-nav-2d-utils ros-humble-nav2-amcl ros-humble-nav2-behavior-tree
  ros-humble-nav2-behaviors ros-humble-nav2-bt-navigator ros-humble-nav2-collision-monitor
  ros-humble-nav2-common ros-humble-nav2-constrained-smoother ros-humble-nav2-controller
  ros-humble-nav2-core ros-humble-nav2-costmap-2d ros-humble-nav2-dwb-controller
  ros-humble-nav2-lifecycle-manager ros-humble-nav2-map-server ros-humble-nav2-mppi-controller
  ros-humble-nav2-msgs ros-humble-nav2-navfn-planner ros-humble-nav2-planner
  ros-humble-nav2-regulated-pure-pursuit-controller ros-humble-nav2-rotation-shim-controller
  ros-humble-nav2-route ros-humble-nav2-rviz-plugins ros-humble-nav2-simple-commander
  ros-humble-nav2-smac-planner ros-humble-nav2-smoother ros-humble-nav2-theta-star-planner
  ros-humble-nav2-util ros-humble-nav2-velocity-smoother ros-humble-nav2-voxel-grid
  ros-humble-nav2-waypoint-follower ros-humble-ompl ros-humble-slam-toolbox ros-humble-smclib xtl-dev
Suggested packages:
  doc-base krb5-doc libbenchmark-tools graphicsmagick-dbg graphicsmagick krb5-user ncurses-doc libnorm-doc
  libomp-14-doc libwmf-0.2-7-gtk libwmf-doc xtensor-doc
The following NEW packages will be installed:
  bzip2-doc comerr-dev graphicsmagick-libmagick-dev-compat krb5-multidev libbenchmark-dev libbenchmark1
  libbsd-dev libbz2-dev libceres-dev libceres2 libgflags-dev libgflags2.2 libgoogle-glog-dev
  libgoogle-glog0v5 libgraphblas-dev libgraphblas6 libgraphics-magick-perl libgraphicsmagick++1-dev
  libgraphicsmagick1-dev libgssrpc4 libkadm5clnt-mit12 libkadm5srv-mit12 libkdb5-10 libkrb5-dev liblcms2-dev
  libldl2 libmd-dev libmongoose2 libnanoflann-dev libncurses-dev libnorm-dev libomp-14-dev libomp-dev
  libomp5-14 libpgm-dev librbio2 libsliplu1 libsodium-dev libspqr2 libsuitesparse-dev libunwind-dev
```

33

[33] Installing Nav2 planners, controllers, lifecycle managers, and supporting libraries.

Docker Desktop installation progress I



34

[34] Docker Desktop installing WSL integration components required for containerized ROS development.

ROS 2 Development Workflow on WSL (Part 1) I

1. Install WSL and Ubuntu
2. Fix networking and configure WSL
3. Update system packages
4. Install ROS 2 Humble
5. Validate environment and middleware
6. Run talker/listener examples
7. Install and run turtlesim

[35]nitial setup stages for building a reliable ROS 2 workspace under Windows Subsystem for Linux.

[35] I

ROS 2 Development Workflow on WSL (Part 2) I

8. Visualize ROS graph with rqt
9. Inspect topics, nodes, services, actions
10. Set `ROS_DOMAIN_ID` and confirm DDS behavior
11. Install Zenoh RMW
12. Install TurtleBot3 and Nav2
13. Install Docker for containerized ROS workflows

[36]dvanced ROS 2 tooling, navigation frameworks, and containerized development under WSL2.

[36] A

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Vielen Dank

für Ihre Aufmerksamkeit

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Quellen

References I


Macenski, Steve u. a. (2020). "The Marathon 2: A Navigation System". In: *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. URL: https://github.com/ros-planning/navigation2.


Macenski, Steven u. a. (2022). "Robot Operating System 2: Design, architecture, and uses in the wild". In: *Science Robotics* 7.66, eabm6074. DOI: 10.1126/scirobotics.abm6074. URL: www.science.org.


Macenski, Steven u. a. (2023). "Impact of ROS 2 Node Composition in Robotic Systems". In: *IEEE Robotics and Autonomous Letters (RA-L)*. DOI: 10.48550/arXiv.2305.09933. URL: https://arxiv.org/abs/2305.09933.


Quigley, Morgan u. a. (2009). "ROS: An open-source robot operating system". In: *Workshops at the IEEE International Conference on Robotics and Automation*. Kobe, Japan.